Faculty of engineering ainshams university

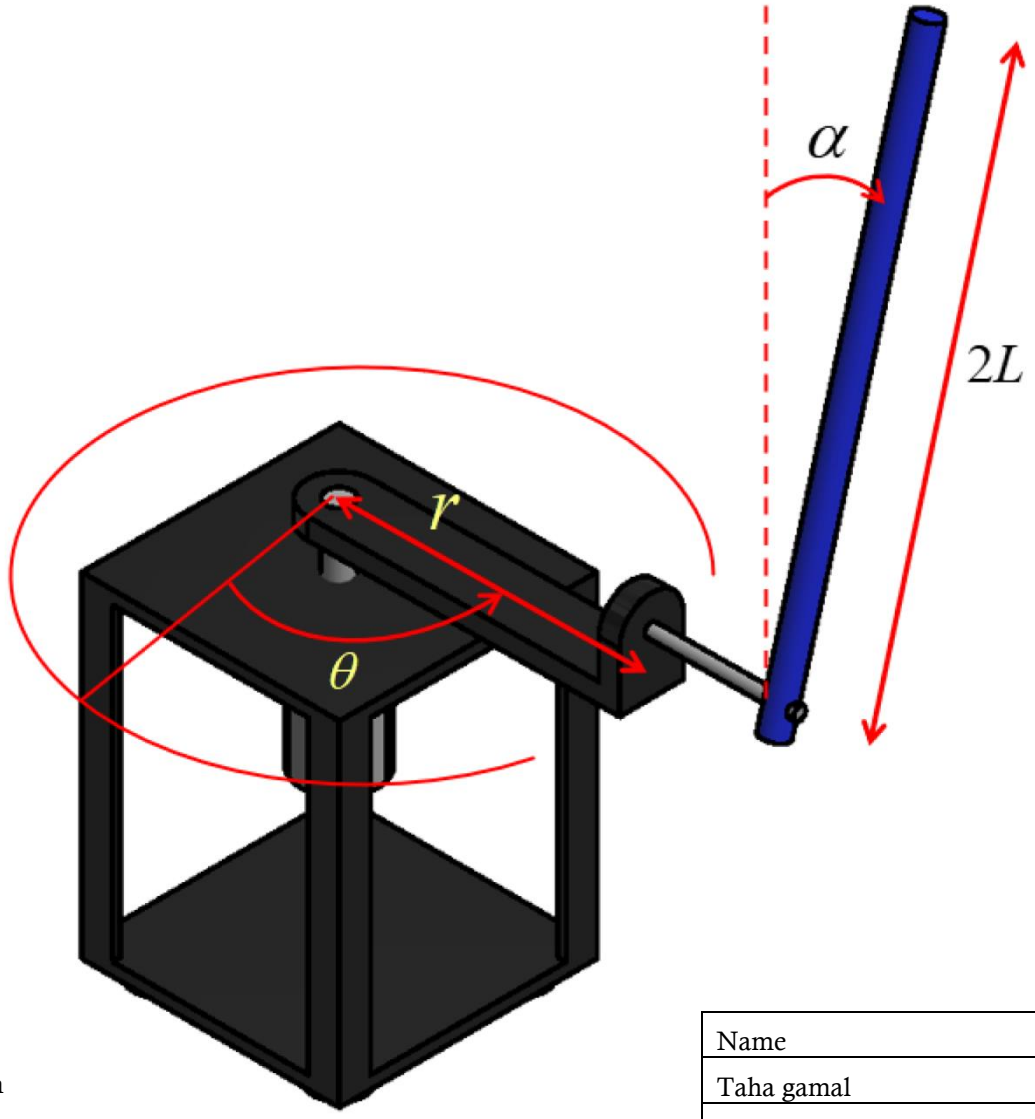
Senior-2 mechatronics

Fall 25

MCT411s

Hybrid control systems

MODELING AND CONTROL OF THE INVERTED ROTARY PENDULUM (FURUATA PENDULUM) TECHNICAL REPORT



Submitted to:

Dr\ Mohamed Ibrahim

Dr\ Mohamed Hamdy

Eng\ Mohab

Eng\ Hesham

Name	id
Taha gamal	2100566
Adham sabry	2100703
Kholoud ahmed	2100712
Sarah nizar	2002325
Suhaila khaled	2001835

TABLE OF CONTENTS

TABLE OF FIGURES

1. ABSTRACT

The Furuta pendulum is a well-known benchmark in the field of underactuated mechanical systems due to its reduced number of control inputs compared to its degrees of freedom, and richly nonlinear behavior. This work addresses the challenge of accurately modeling and controlling such a system without relying on traditional linearization techniques. In contrast to the common approach based on Lagrangian analytical modeling and state–space linearization, we used a methodology that integrates a high-fidelity multibody model developed in Simscape Multibody (MATLAB), capturing the complete nonlinear dynamics of the system. The multibody model includes all geometric, inertial, and joint parameters of the physical hardware and interfaces directly with Simulink, enabling realistic simulation and control integration. For the furuta pendulum to stay at the upright position, we linearize about that position for our linear controller, the lqr controller. But what if it started from the bottom? We need another controller and that is the swing up controller.

Keywords: Furuta pendulum; multibody dynamics; nonlinear control; hybrid control;

2. INTRODUCTION

2.1 FURUTA PENDULUM

The rotary pendulum is also known as Furuta′s pendulum. Furuta was the inventor, and has been studied by many authors such as Furuta himself [1] (see also [2–4] among others). It is a non-linear underactuated mechanical system that is unstable at the desired upright position. In fact, it shows, mainly, two different and very interesting control problems. The first one is to swing the pendulum up from the rest (down) to the upright (up) position, which is commonly solved with energy control strategies. Once the pendulum is close to the desired upright position, at low speed, a stabilization or balancing strategy “catches” it there. Other control

problems are also quite interesting as the stabilization of autonomous oscillations and control through bifurcation analysis.

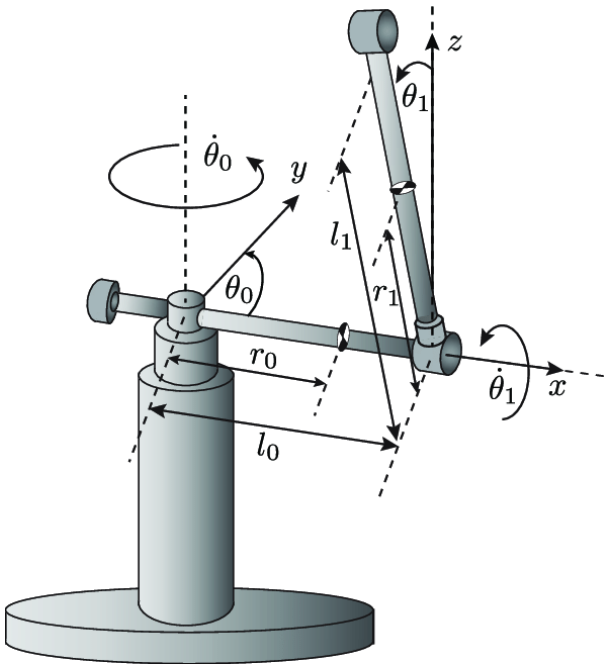


Figure 1 furuta pendulum

2.2 SWING-UP CONTROLLER

A strategy to transition the pendulum from a stable downward position to the unstable upright equilibrium. This involves designing a controller that can efficiently transfer energy to the pendulum, overcoming the gravitational potential barrier and positioning it for stabilization.

The primary challenge was transitioning the pendulum from its stable downward position ($\theta \approx 0$) to the unstable upright position ($\theta \approx \pi$). This required carefully managing the system's energy.

2.3 LINEAR QUADRATIC REGUALTOR (LQR)

The theory of optimal control is concerned with operating a dynamic system at minimum cost. The case where the system dynamics are described by a set of linear differential equations and the cost is described by a quadratic function is called the LQ problem. One of the main results in the theory is that the solution is provided by the linear–quadratic regulator (LQR), a feedback controller whose equations are given below.

The settings of a (regulating) controller governing either a machine or process (like an airplane or chemical reactor) are found by using a mathematical algorithm that minimizes a cost function with weighting factors supplied by the operator. The cost function is often defined as a sum of the deviations of key measurements, like altitude or process temperature, from their desired values. The algorithm thus finds those controller settings that minimize undesired deviations. The magnitude of the control action itself may also be included in the cost function.

The LQR algorithm reduces the amount of work done by the control systems engineer to optimize the controller. However, the engineer still needs to specify the cost function parameters, and compare the results with the specified design goals. Often this means that controller construction will be an iterative process in which the engineer judges the "optimal" controllers produced through simulation and then adjusts the parameters to produce a controller more consistent with design goals.

The LQR algorithm is essentially an automated way of finding an appropriate state-feedback controller. As such, it is not uncommon for control engineers to prefer alternative methods, like full state feedback, also known as pole placement, in which there is a clearer relationship between controller parameters and controller behavior. Difficulty in finding the right weighting factors limits the application of the LQR based controller synthesis.

3. OBJECTIVES

- Introducing realistic uncertainties by adding randomness and noise. By simulating sensor inaccuracies, actuator noise, and external disturbances, we can evaluate the controller's performance in scenarios that mimic real world operational environments.
- Implementing and evaluating two control methodologies:
    1. Swing-Up Control: A strategy to transition the pendulum from a stable downward position to the unstable upright equilibrium. This involves designing a controller that can efficiently transfer energy to the pendulum, overcoming the gravitational potential barrier and positioning it for stabilization.
    2. Linear Quadratic Regulator (LQR): A control approach to stabilize the pendulum around the upright position with minimal oscillations and control effort. The LQR is designed by linearizing thesystem around the desired equilibrium point and determining the optimal feedback gains that minimize a defined cost function balancing state deviations and control effort.

4. METHODOLOGY

4.1 MODELING

4.1.1 ENCODER OF THE DC MOTOR

Determining the ppr :

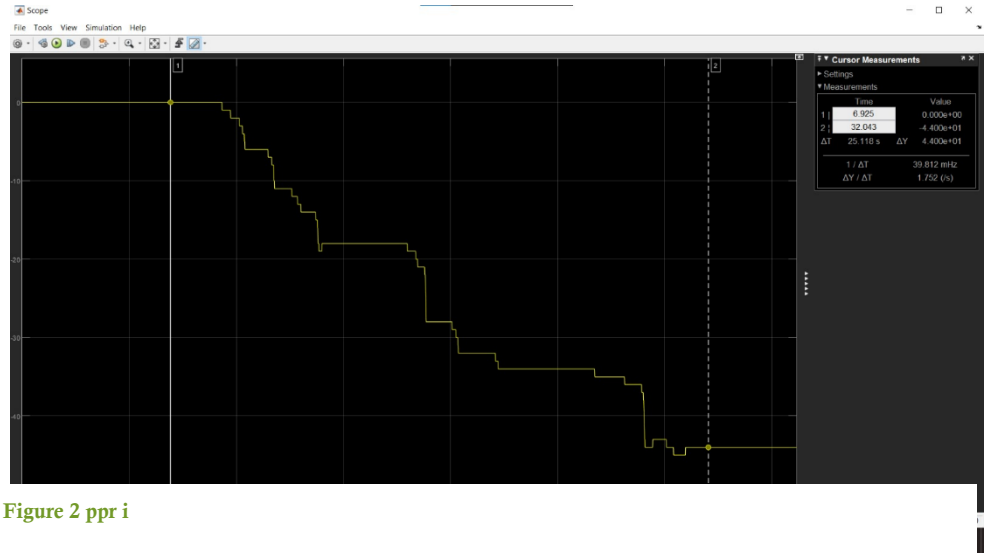


Figure 2 ppr i

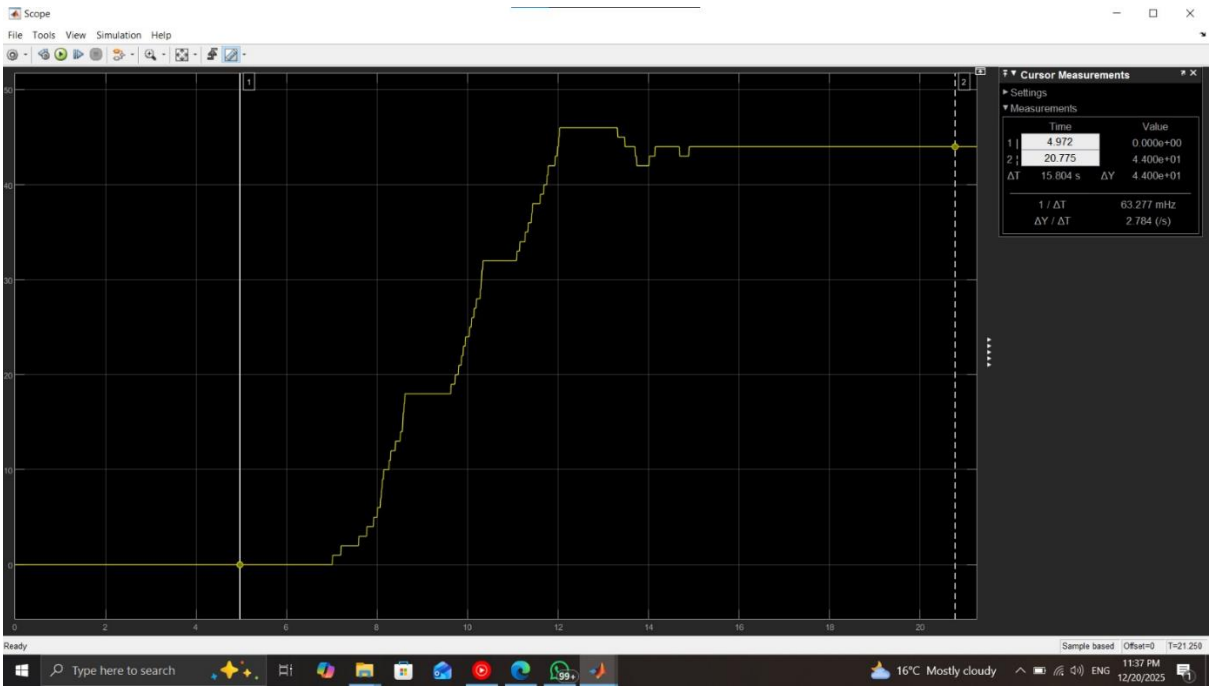


Figure 3 ppr ii

4.1.2 DC MOTOR MODEL FOR PARAMETER ESTIMATION

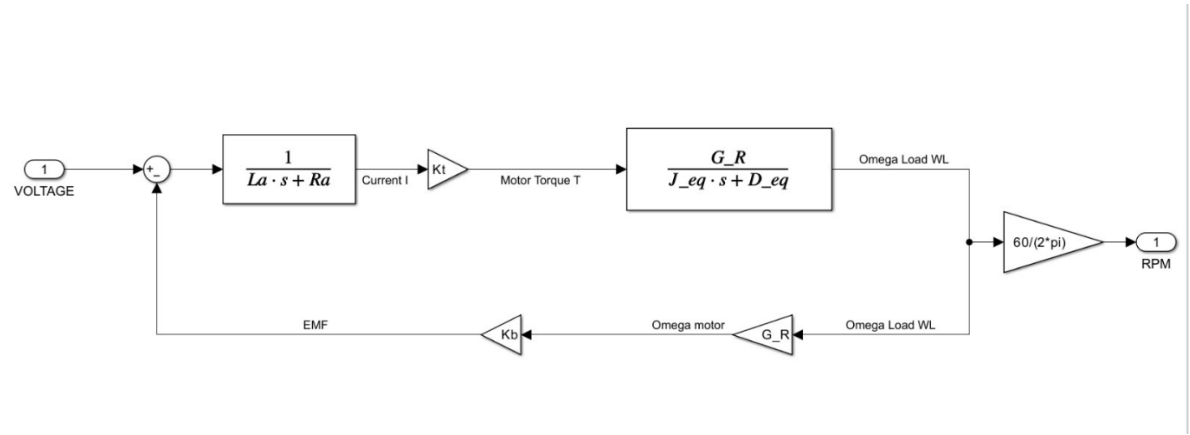


Figure 4 dc motor model

4.1.2.1 PARAMETER ESTIMATION OF THE DC MOTOR

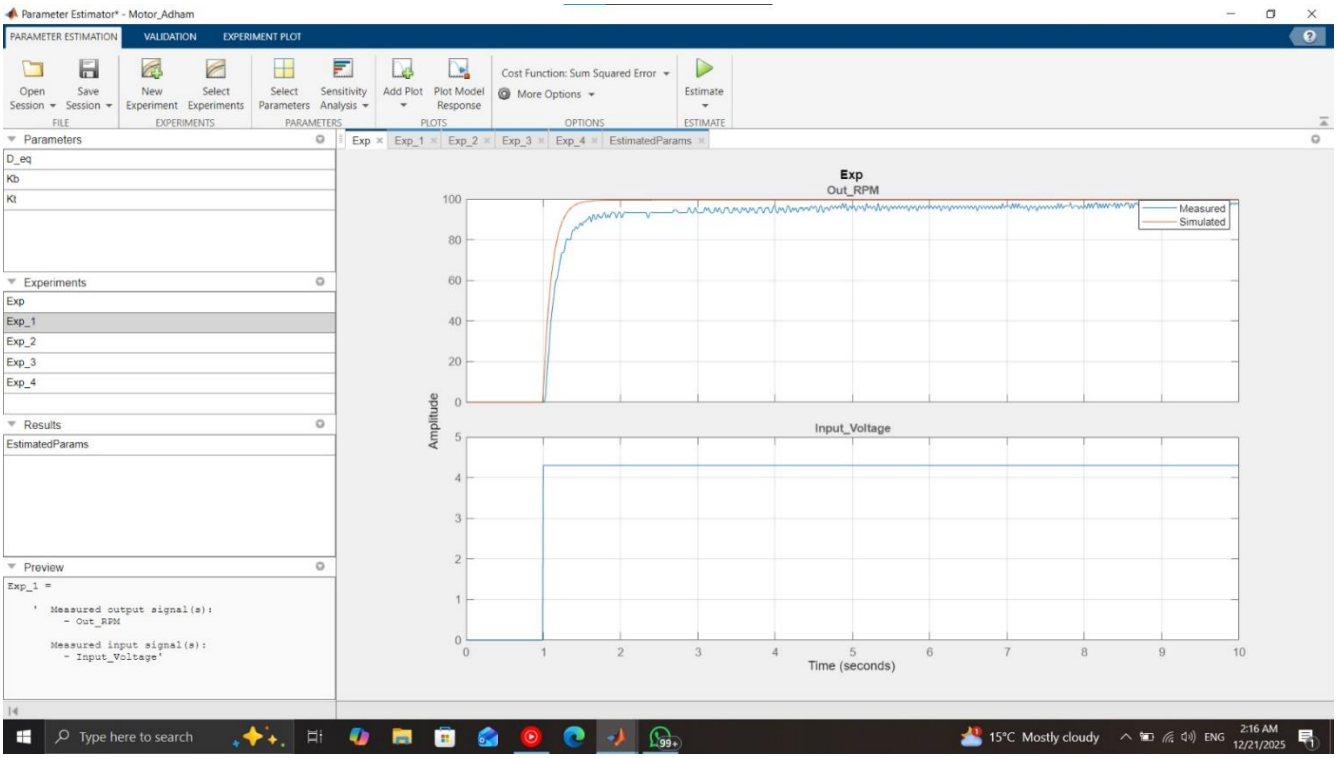


Figure 5 exp 1

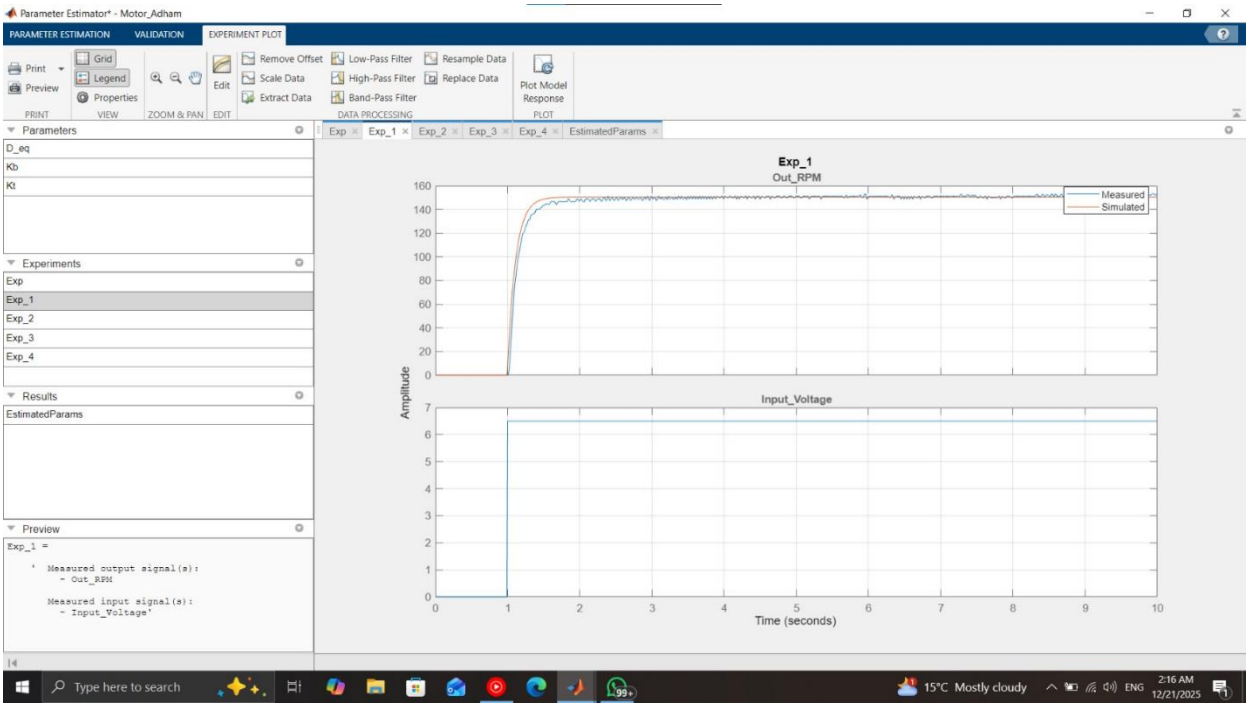


Figure 6 exp 2

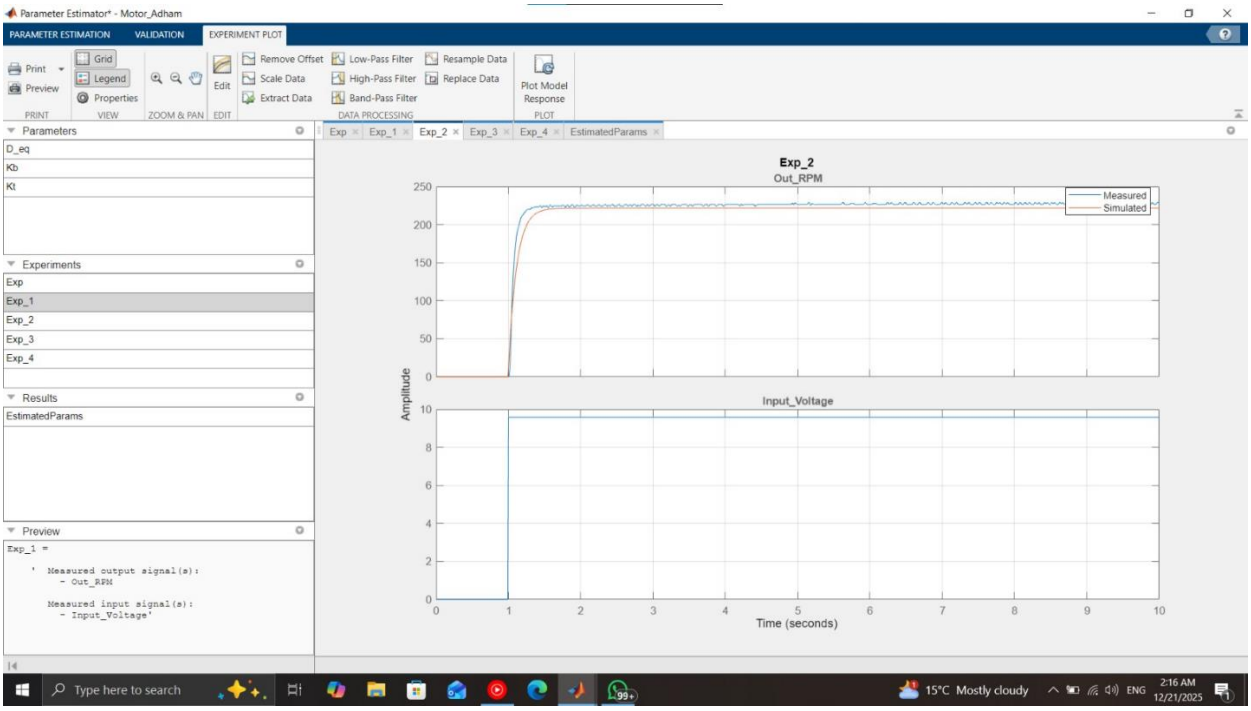


Figure 7 exp 3

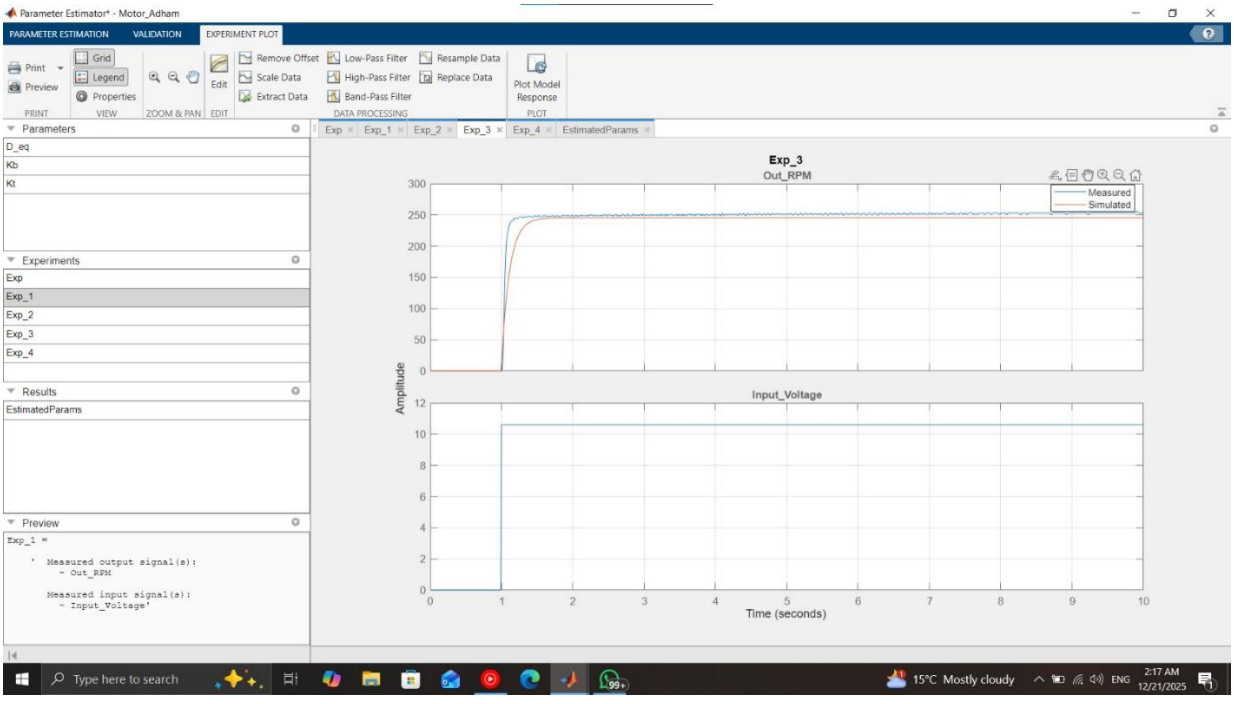


Figure 8 exp 4

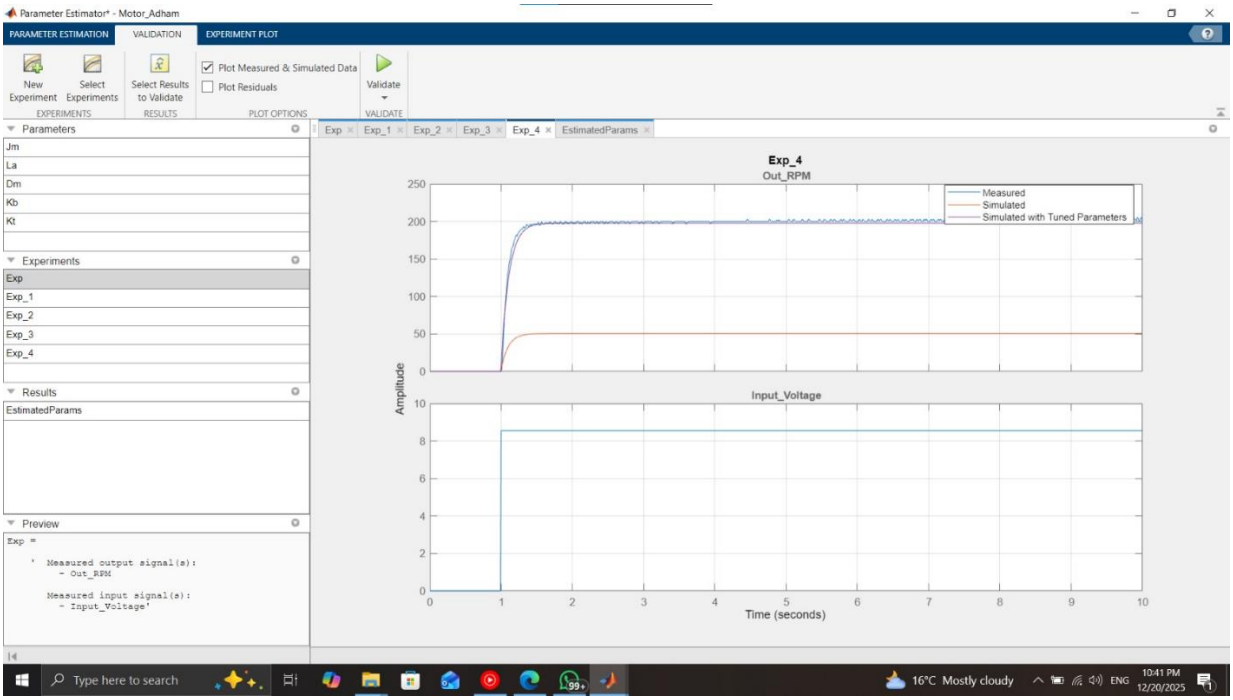


Figure 9 validation

4.1.3 LINEARIZATION

From the paper : [2010_ICMA_ModelingandControlofaRotaryInvertedPendulumUsingVariousMethods.pdf - Google Drive]

$$\begin{bmatrix}\dot{\theta}\\ \dot{\alpha}\\ \ddot{\theta}\\ \ddot{\alpha}\end{bmatrix} = \begin{bmatrix}0 & 0 & 1 & 0\\ 0 & 0 & 0 & 1\\ 0 & \frac{bd}{E} & \frac{-cG}{E} & 0\\ 0 & \frac{qd}{E} & \frac{-bG}{E} & 0\end{bmatrix}\begin{bmatrix}\theta\\ \alpha\\ \dot{\theta}\\ \dot{\alpha}\end{bmatrix} + \begin{bmatrix}0\\ 0\\ c\frac{\eta_m\eta_g K_t K_g}{R_m E}\\ b\frac{\eta_m\eta_g K_t K_g}{R_m E}\end{bmatrix} V_m \qquad (16)$$

Here, $a = J_{eq} + mr^2$, $b = mLr$, $c = 4/3mL^2$, $d = mgL$, $E = ac - b^2$, $G = \frac{\eta_m\eta_g K_t K_m K_g^2 - B_{eq}R_m)}{R_m}$. The following table

Figure 10 state space

Symbol	Description
K_t	Motor Torque Constant
K_m	Back EMF Constant
R_m	Armature Resistance
K_g	SRV02 system gear ratio (motor->load)
η_m	Motor efficiency
η_g	Gearbox efficiency
B_{eq}	Equivalent viscous damping coefficient
J_{eq}	Equivalent moment of inertia at the load

Figure 11 parameters

4.1.4 LINEAR MODEL OF THE FURUTA PENDULUM

State space:

4.2 CONTROLLERS DESIGN

4.2.1 SWING-UP CONTROLLER

The principle of energy-based swing-up is to inject energy into the pendulum link until it reaches the specific "energy level" required to stay upright. Instead of controlling the angle directly (which is difficult when it's hanging down), we control the **total mechanical energy** of the pendulum.

1. Principle of Operation

The primary challenge of a Furuta Pendulum is that the motor acts on the **horizontal arm**, but we need to move the **vertical pendulum** to an upright position. A standard linear controller (like LQR or PID) only works when the pendulum is already near the top.

The **Energy-Based Controller** solves this by treating the pendulum as a system where we "pump" energy into it until it reaches the energy level required to stand upright. Once the pendulum reaches a specific energy threshold and angle, the system switches to a stabilization controller.

2. Design Equations

A. System Energy (E) The total mechanical energy of the pendulum link is the sum of its kinetic and potential energy. Using the parameters **c** and **d** from your model:

E = 0.5 * c * (alpha_dot)^2 + d * (cos(alpha) - 1)

Where:

- **c**: Total pendulum inertia (Jp + mp * Lp^2)
- **d**: Gravity term (mp * g * Lp)

- **alpha**: Pendulum angle (0 is upright, 180 degrees is hanging down)
- **alpha_dot**: Angular velocity of the pendulum

B. Target Energy (E0) In the upright, balanced position (alpha = 0, alpha_dot = 0), the potential energy is zero and kinetic energy is zero. Therefore, the target energy is:

E0 = 0

C. Energy Dynamics To understand how the arm's motion affects the pendulum's energy, we look at the derivative of the energy. The change in energy is driven by the horizontal arm's acceleration (theta_ddot):

Change in Energy = -(b * cos(alpha) * alpha_dot) * theta_ddot

This shows that we can increase or decrease energy by controlling the acceleration of the horizontal arm.

3. The Control Law

To drive the pendulum energy (E) toward the target (E0), we use the **Astrom-Furuta** energy control law. The desired acceleration for the horizontal arm (theta_ddot_ref) is calculated as:

theta_ddot_ref = k * (E - E0) * sign(alpha_dot * cos(alpha))

Where:

- **k**: A positive control gain that determines the speed of the swing-up.

4. Implementation and Switching Logic

In a real-world system, you cannot use the energy controller indefinitely because it will cause the pendulum to swing past the upright position. A **hybrid control strategy** is required:

Phase 1: Swing-Up If the pendulum is outside the "catch" zone (for example, alpha is greater than 20 degrees from the top): **Motor Voltage = Gain_Swing * (E - E0) * sign(alpha_dot * cos(alpha))**

Phase 2: Stabilization (The "Catch") If the pendulum enters the "catch" zone (alpha is less than 20 degrees and alpha_dot is low): **Motor Voltage = -(K1 * theta + K2 * alpha + K3 * theta_dot + K4 * alpha_dot)** *(This is your LQR or PID control)*

4.2.2 LQR CONTROLLER

1. The Principle of LQR

The Linear Quadratic Regulator (LQR) is an "optimal" control strategy. Its goal is to stabilize the pendulum in the upright position while balancing two competing priorities:

1. **Performance:** Minimizing the error (how far the pendulum is from upright).
2. **Effort:** Minimizing the control energy (how much voltage the motor uses).

It is called **Full-State Feedback** because the controller requires knowledge of all four "states" of the system at every moment to calculate the correct motor voltage.

2. The State Vector

To control the pendulum, we track four specific variables, organized into a state vector **x**:

- **theta**: Horizontal arm angle.
- **alpha**: Vertical pendulum angle (0 is perfectly upright).
- **theta_dot**: Angular velocity of the arm.
- **alpha_dot**: Angular velocity of the pendulum.

State Vector x = [theta, alpha, theta_dot, alpha_dot]

3. The Linear State-Space Model

LQR is a linear control method, but the Furuta Pendulum is naturally non-linear. Therefore, we use a linearized model that is valid only when the pendulum is near the upright position:

x_dot = A * x + B * u

Where:

- **A**: The System Matrix (defines how the pendulum moves naturally based on the parameters a, b, c, d).
- **B**: The Input Matrix (defines how the motor voltage affects the system).
- **u**: The Control Input (Motor Voltage).

4. The Cost Function (The "Quadratic" part)

The "Optimal" part of LQR comes from minimizing a mathematical cost function, usually labeled **J**. The designer chooses two weights:

J = Sum of (State_Error * Q * State_Error) + (Control_Effort * R * Control_Effort)

- **Q Matrix**: A 4x4 matrix that defines how much you care about each state. (e.g., if you want the pendulum to stay very still, you increase the weight for alpha).
- **R Matrix**: A single value that defines how much you want to save motor energy. (A large R makes the controller "lazy" but saves power; a small R makes the controller very aggressive).

5. The Design Equations

A. Calculating the Gain (K) The LQR algorithm solves a complex matrix equation (the Algebraic Riccati Equation) to find the optimal feedback gain vector **K**. In MATLAB, this is done using the command: **K = lqr(A, B, Q, R)**

The result is a row vector with four values: **K = [k1, k2, k3, k4]**.

B. The Control Law Once the gains are found, the motor voltage (**u**) is calculated by multiplying the gains by the current states:

u = -(k1 * theta + k2 * alpha + k3 * theta_dot + k4 * alpha_dot)

This is the "Full-State Feedback" equation. It constantly adjusts the motor voltage to push all four states back toward zero (the upright balanced state).

5. CONTROLLERS IMPLEMENTATION

5.1 SWING-UP CONTROLLER

Into a matlab function :

```
function u_swing = fcn(theta1, theta1_dot)

% Parameters

g = 9.81;

l = 0.1;

m =0.012;

% --- CHANGE 1: HUGE GAIN ---

k_swing = 1000;


% Energy Calculation (Positive PE)

PE = m * g * l * cos(theta1);

KE = 0.5 * m * l^2 * theta1_dot^2;

E_total = PE + KE;

E_ref = m * g * l;

E_error = E_ref - E_total;


% --- CHANGE 2: HARD KICK ---

% If moving very slowly AND at the bottom...

if abs(theta1_dot) < 0.2 && abs(cos(theta1) + 1) < 0.2

    % Full power kick to wake up the heavy arm
```

```
    u_swing = 8;
else

    % Normal Swing Control
    % Note: Try positive k_swing first. If it acts like a brake, add a negative sign: -k_swing
    u_raw = -k_swing * E_error * sign(theta1_dot * cos(theta1));

    % --- CHANGE 3: FULL 12V LIMIT ---
    u_swing = min(max(u_raw,-12),12);
end
```

5.2 LQR CONTROLLER

LQR SCRIPT:

```
A=linsys35.A
B=linsys35.B
C=linsys35.C
D=linsys35.D
q=diag([ 6,50 ,55 ,50,0]);
%      [  theta   thetadot alpha  alphadot tf]

r_eq=0.01;
k=lqr(A,B,q,r_eq);
k_theta = k(1)
k_alpha=k(3)
k_thetadot=k(2)
k_alphadot=k(4)
KF = [k_theta, k_thetadot, k_alpha, k_alphadot]
```

Figure 12 LQR

Simulink blocks :

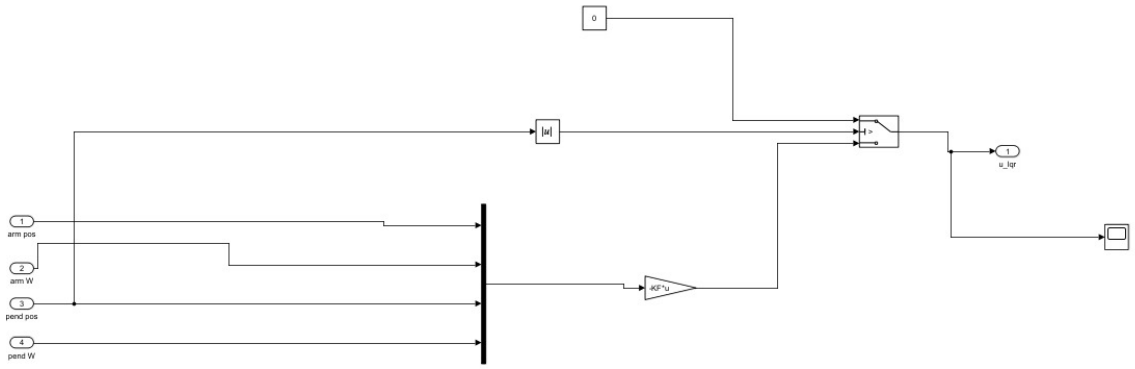


Figure 13 blocks of LQR controller

6. CONTROLLERS + PLANT

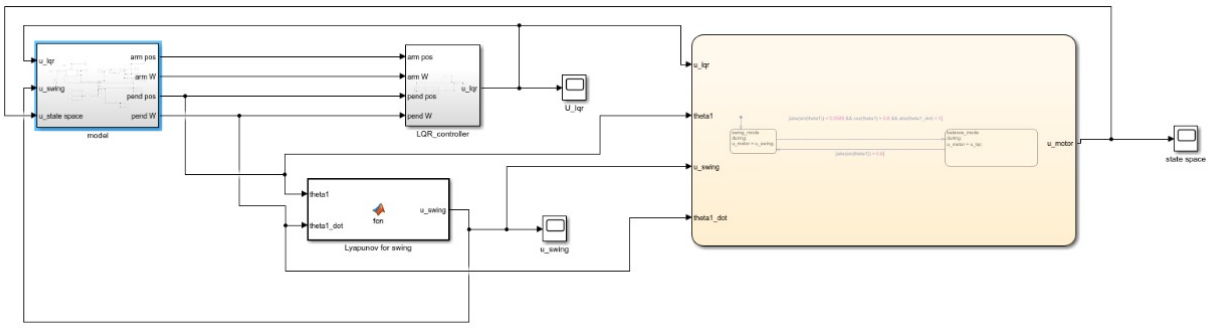


Figure 14 the 2 controllers with the model

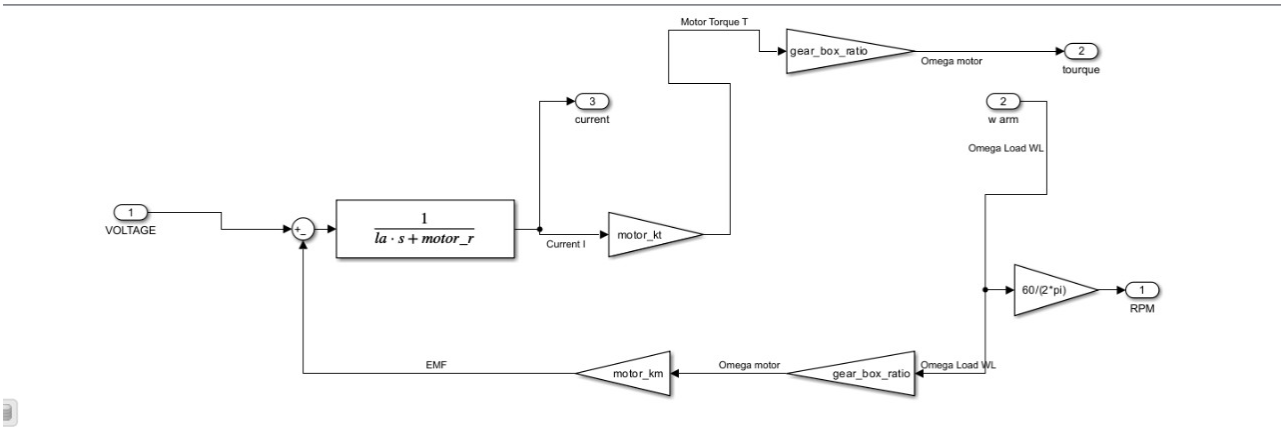


Figure 15 motor model for simulation

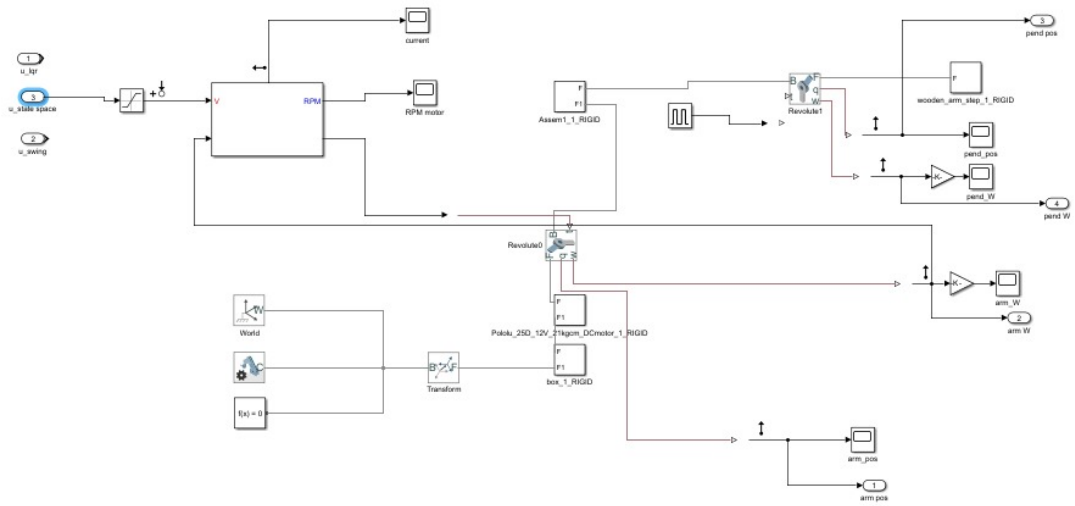


Figure 16 simscape multibody model exported from SolidWorks

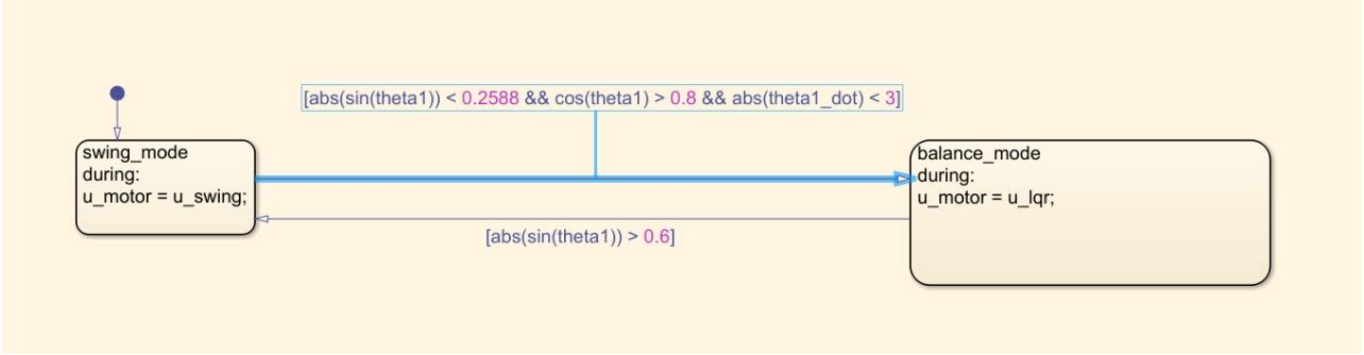


Figure 17 stateflow chart for simulations

7. HARDWARE

7.1 COMPONENTS USED

- Dc moor with encoder
- 2 hex couplers
- Arduino mega
- Motor driver
- Encoder
- 3d printed part
- Wooden box for casing
- Wooden pendulum
- Wires
- Bread board

7.2 REAL-LIFE PICTURE OF THE FURUTA PENDULUM

Figure 18 real life picture

8. REAL FURUTA PENDULUM RESULTS

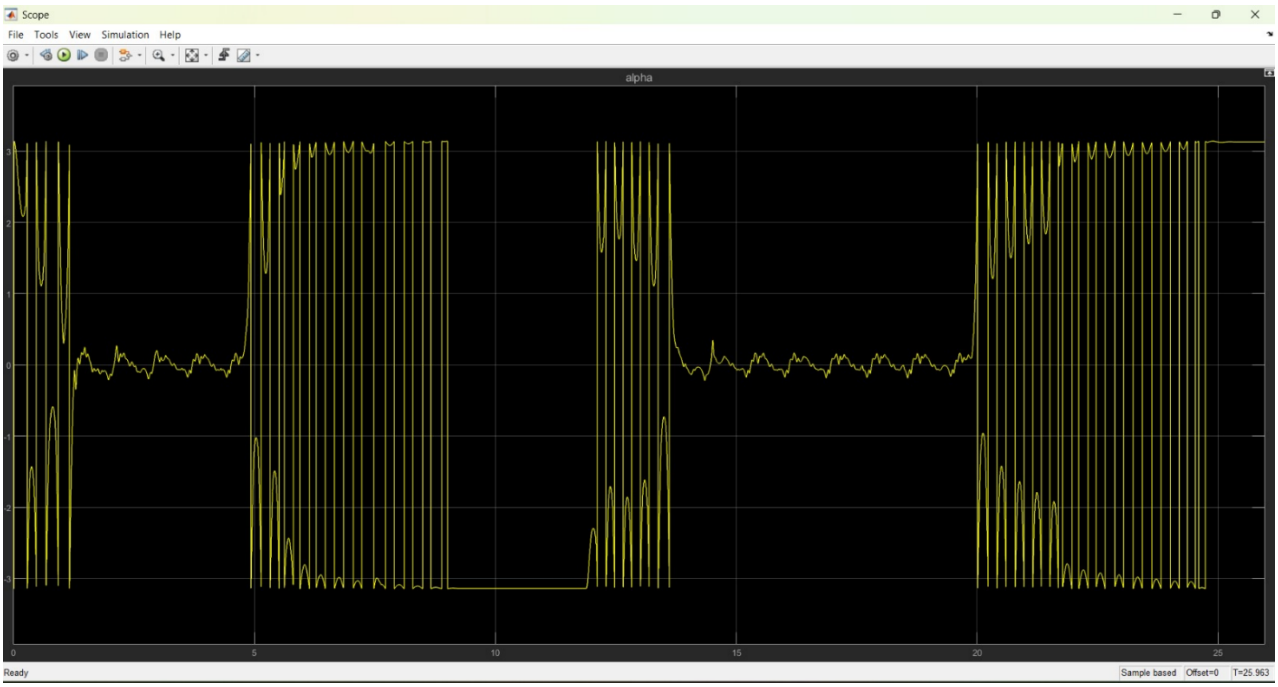


Figure 19 pendulum angle

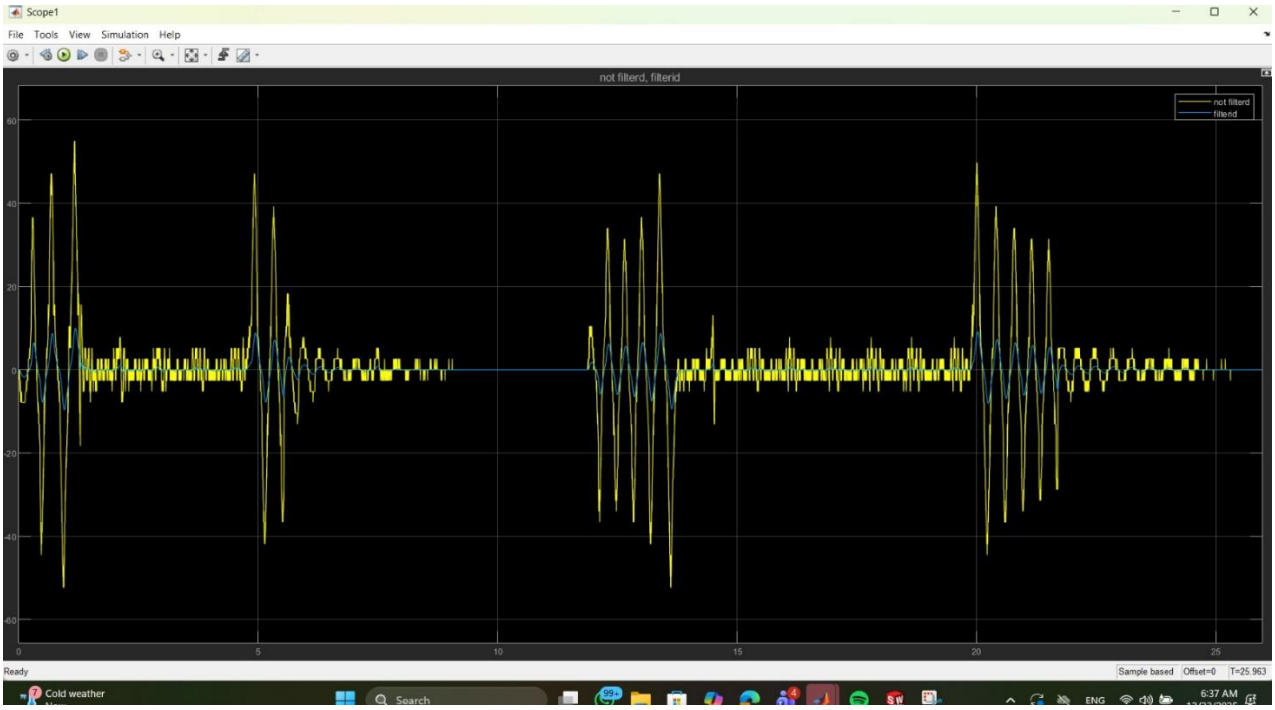


Figure 20 pendulum angular velocity

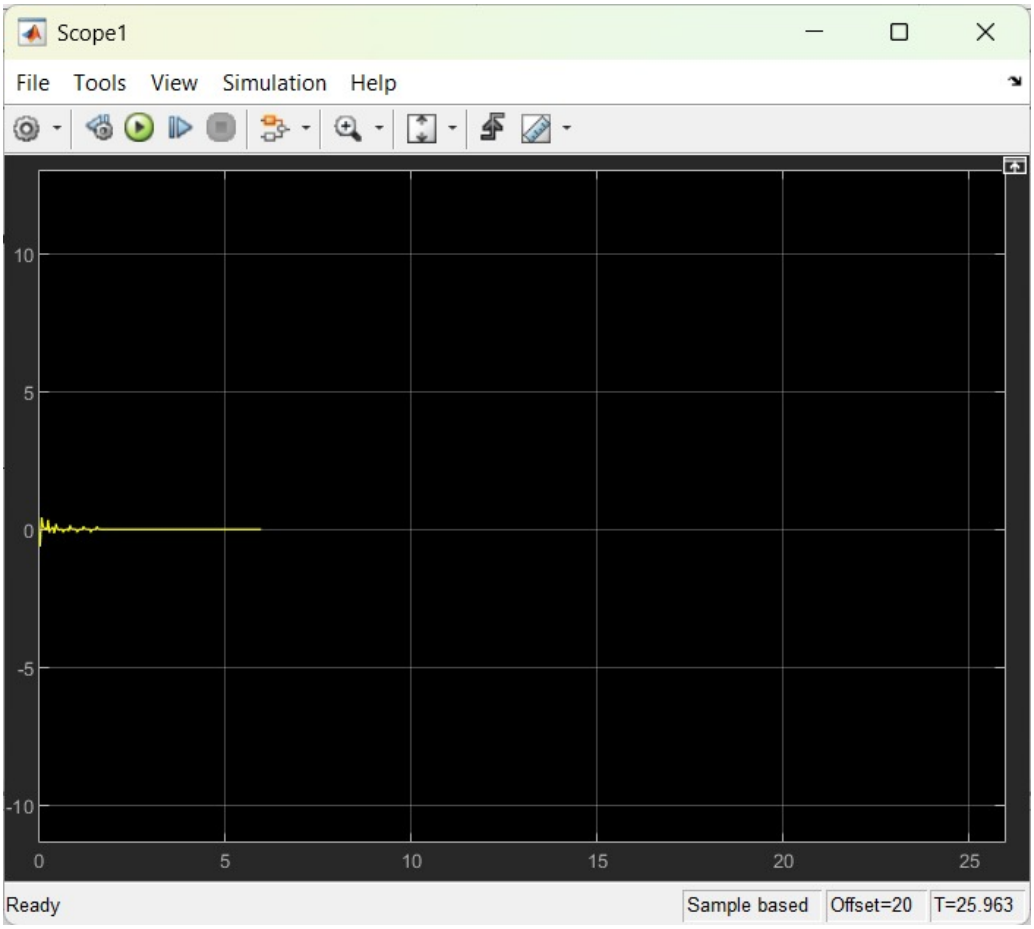


Figure 21 motor angle

Figure 22 motor angular velocity

9. SIMULATION RESULTS VS REAL FURUTA PENDULUM RESULTS

10. CHALLENGES AND LESSONS LEARNED

1. Getting the correct geometry for the hardware system to work appropriately.
2. Choosing the suitable motor for our design
3. considering: gearbox and gearbox efficiency,…..
4. Tuning gain of swing up.

*Lessons Learned:-

1. The Importance of High-Fidelity Modeling One of the most critical lessons was that a controller is only as good as the model it is built upon. Learning to calculate the "Equivalent Inertia" (Jeq) taught us that motor rotor inertia, gearbox ratios, and mass distribution are not just secondary details—they are fundamental to the system's behavior. Using the Euler-Lagrange method allowed for a structured way to capture these dynamics.

2. Managing Underactuated Systems The project highlighted the unique challenge of underactuated systems: we only have one "input" (the motor voltage at the arm) but must control two "degrees of freedom" (the arm angle and the pendulum angle). This taught us that control is often about managing energy and momentum, rather than just forcing a single variable to a setpoint.

3. Linearization and Its Limits Implementing the LQR controller demonstrated the power of linear control theory, but also its limitations. We learned that while a linear model (A and B matrices) works perfectly for small angles near the top, it fails completely when the pendulum falls too far. This necessitated the "Switching Logic" between the Swing-up and the LQR.

4. Energy-Based Control Design Designing the swing-up controller introduced the concept of Lyapunov-like energy methods. Instead of trying to fix the angle, we learned to "pump" energy into the system. This shifted our perspective from traditional PID-style error-tracking to energy-state management, which is a key concept in modern robotics.

5. Practical Hardware Constraints Finally, the project provided insight into "real-world" engineering. Concepts like motor voltage saturation, gearbox friction, and sensor noise (in theta_dot and alpha_dot) required practical adjustments that are not usually found in textbook equations. Balancing these hardware realities with theoretical math was a vital part of the learning process.

11. CONCLUSION

The Furuta Pendulum project successfully demonstrated the complex relationship between mechanical design and advanced control theory. By taking a highly unstable, non-linear, and underactuated system—where a horizontal arm must move to balance a vertical rod—the project provided a platform to apply the full "Control Design Cycle": from mathematical modeling using Euler-Lagrange equations to the implementation of a hybrid control strategy using Energy-Based Swing-up and Linear Quadratic Regulator (LQR) stabilization.

The final system was able to successfully transition from a state of rest (hanging down) to a stable, upright equilibrium. This was achieved by accurately calculating the system's equivalent inertia (Jeq), accounting for motor dynamics, and carefully tuning the control gains to overcome physical disturbances and friction.

12. REFERENCES

Gäfvert Magnus (1998) "Modelling the Furuta Pendulum," Technical Reports; TFRT.

(PDF) Furuta's Pendulum: A Conservative nonlinear model for theory and practise. (n.d.). https://www.researchgate.net/publication/42387157_Furuta's_Pendulum_A_Conservative_Nonlinear_Model_for_Theory_and_Practise

https://www.researchgate.net/publication/236619208_Swing-Up_Methods_For_Inverted_Pendulum

(PDF) swing-up methods for inverted pendulum. (n.d.-b). https://www.researchgate.net/publication/236619208_Swing-Up_Methods_For_Inverted_Pendulum

Bajrami, X., Pajaziti, A., Likaj, R., Shala, A., Berisha, R., & Bruqi, M. (2021, August 13). *Control theory application for swing up and stabilisation of rotating inverted pendulum*. MDPI. https://www.mdpi.com/2073-8994/13/8/1491

Linearization manager. Manage linear analysis points and block linearizations - MATLAB. (n.d.). https://www.mathworks.com/help/slcontrol/ug/linearizationmanager-app.html

(No date) *(PDF) modeling, design and implementation control of swing up and swinging of an inverted rotary pendulum*. Available at: https://www.researchgate.net/publication/234818854_MODELING_DESIGN_AND_IMPLEMENTATION_CONTROL_OF_SWING_UP_AND_SWINGING_OF_AN_INVERTED_ROTARY_PENDULUM (Accessed: 23 December 2025).

13. DRIVE LINK

hybrid____ - Google Drive